TECNOLÓGICO NACIONAL DE MEXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
ENERO - JUNIO 2026

NOMBRE DE LA CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

NOMBRE DE LA MATERIA:
PATRONES DE DISEÑO DE SOFTWARE

TÍTULO DEL TRABAJO:
PATRÓN DE DISEÑO ABSTRACT FACTORY

UNIDAD A EVALUAR:
UNIDAD 2

NOMBRE DEL ALUMNO:
ACOSTA MARTINEZ MOISES - 22210278
GARCÍA CALDERÓN FERNANDO - 22210306
HERNÁNDEZ RAMÍREZ LIXBETH DENISSE - 22210313
RODRIGUEZ MORALES SUSANA ABIGAIL - 22210346

NOMBRE DEL MAESTRO:
MARIBEL GUERRERO LUIS

Índice

Fábrica Abstracta 3
Diagrama UML 3
Interfaz 4
- Pantalla principal 4
- Menú 4
- Platillo 5

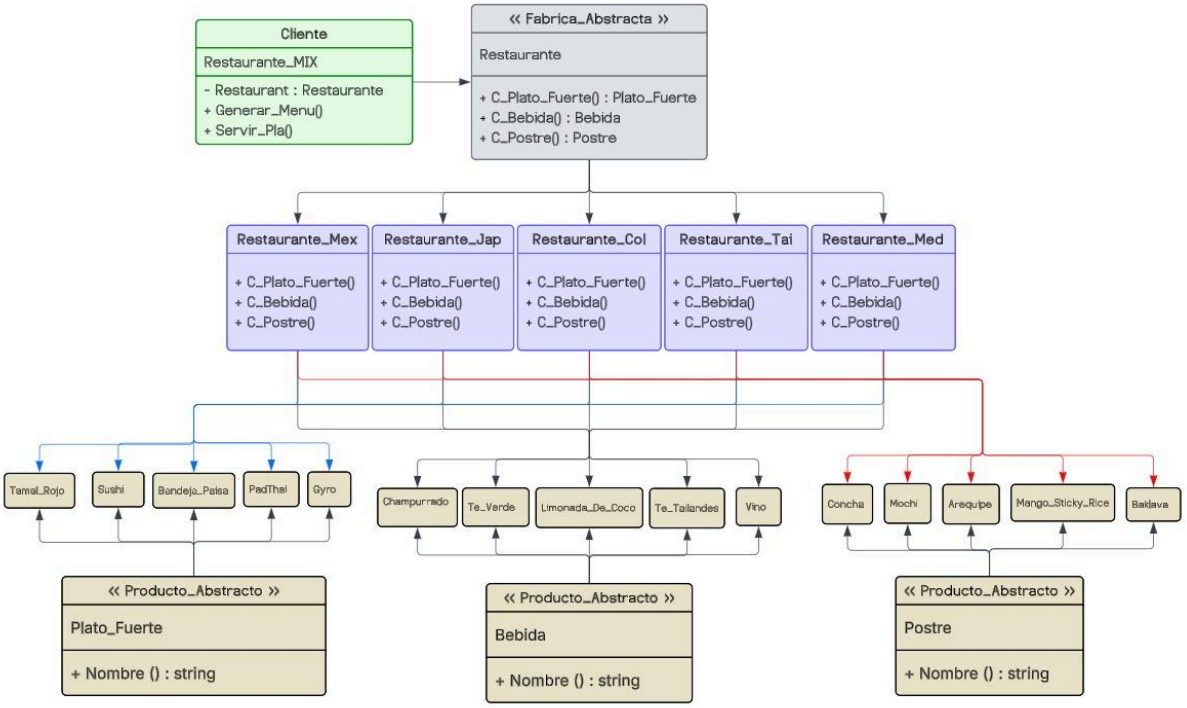
Código 6
- Pantalla principal 6
- Menú 7
- Servicio 11
- Despedida 13

Pruebas 14
Conclusión 15

Fábrica Abstracta

En este proyectó se utilizó el patrón Abstract Factory para representar un restaurante. El patrón permite mostrar las diferentes familias de productos como platillos, bebidas y postres, donde en cada clase concreta, como de comida mexicana, japonesa, tailandesa, etc, contiene un propio conjunto de productos, así formando un menú completo de cada estilo de comida, donde el cliente no va a saber a qué fábrica en especial genera su producto.

Diagrama UML



Interfaz

Pantalla principal

Fig.1: Se muestra la opción de abrir el menú en el lado izquierdo y cerrar el programa en la parte superior derecha con la X.

Menú

Fig.2: El cliente elige el tipo de comida que desee, con la flecha puede regresar a la pantalla principal o cerrar el programa.

Platillo

Fig.3: Aquí se muestra el platillo, la bebida y el postre que pertenece al estilo de comida elegido por el cliente, bien puede volver al inicio o seleccionar otro platillo con los botones en la parte inferior derecha o cerrar del programa.

Código

Pantalla principal

```
namespace Restaurante_Mix
{
    6 referencias
    public partial class Entrada : Form
    {
        2 referencias
        public Entrada()
        {
            InitializeComponent();
        }

        1 referencia
        private void Entrada_Load(object sender, EventArgs e)
        {

        }

        1 referencia
        private void button1_Click(object sender, EventArgs e)
        {
            Menu Menu1= new Menu();
            Menu1.Show();
            this.Hide();
        }

        1 referencia
        private void SALIR_Click(object sender, EventArgs e)
        {
            Application.Exit();
        }
    }
}
```

Menú

```
namespace Restaurante_Mix
{
    12 referencias
    public partial class Menu : Form
    {
        // PRODUCTOS ABSTRACTOS
        12 referencias
        public abstract class Plato_Fuerte
        {
            6 referencias
            public abstract string Nombre();
        }

        12 referencias
        public abstract class Bebida
        {
            6 referencias
            public abstract string Nombre();
        }

        12 referencias
        public abstract class Postre
        {
            6 referencias
            public abstract string Nombre();
        }

        // PRODUCTOS CONCRETOS
```

```
        // PRODUCTOS CONCRETOS
        1 referencia
        class Tamal_Rojo : Plato_Fuerte { public override string Nombre() => "Tamal Rojo"; }
        1 referencia
        class Sushi : Plato_Fuerte { public override string Nombre() => "Sushi"; }
        1 referencia
        class Bandeja_Paisa : Plato_Fuerte { public override string Nombre() => "Bandeja Paisa"; }
        1 referencia
        class PadThai : Plato_Fuerte { public override string Nombre() => "Pad Thai"; }
        1 referencia
        class Gyro : Plato_Fuerte { public override string Nombre() => "Gyro"; }

        1 referencia
        class Champurrado : Bebida { public override string Nombre() => "Champurrado"; }
        1 referencia
        class Te_Verde : Bebida { public override string Nombre() => "Te Verde"; }
        1 referencia
        class Limonada_De_Coco : Bebida { public override string Nombre() => "Limonada de Coco"; }
        1 referencia
        class Te_Tailandes : Bebida { public override string Nombre() => "Te Tailandes"; }
        1 referencia
        class Vino : Bebida { public override string Nombre() => "Vino"; }

        1 referencia
        class Concha : Postre { public override string Nombre() => "Concha"; }
        1 referencia
        class Mochi : Postre { public override string Nombre() => "Mochi"; }
        1 referencia
        class Arequipe : Postre { public override string Nombre() => "Arequipe"; }
        1 referencia
        class Mango_Sticky_Rice : Postre { public override string Nombre() => "Mango Sticky Rice"; }
        1 referencia
        class Baklava : Postre { public override string Nombre() => "Baklava"; }

        // FABRICA ABSTRACTA
```

```
// FABRICA ABSTRACTA
8 referencias
public abstract class Restaurante
{
    6 referencias
    public abstract Plato_Fuerte C_Plato_Fuerte();
    6 referencias
    public abstract Bebida C_Bebida();
    6 referencias
    public abstract Postre C_Postre();
}

1 referencia
public class Restaurante_Mex : Restaurante
{
    2 referencias
    public override Plato_Fuerte C_Plato_Fuerte() => new Tamal_Rojo();
    2 referencias
    public override Bebida C_Bebida() => new Champurrado();
    2 referencias
    public override Postre C_Postre() => new Concha();
}

1 referencia
public class Restaurante_Jap : Restaurante
{
    2 referencias
    public override Plato_Fuerte C_Plato_Fuerte() => new Sushi();
    2 referencias
    public override Bebida C_Bebida() => new Te_Verde();
    2 referencias
    public override Postre C_Postre() => new Mochi();
}

1 referencia
class Restaurante_Col : Restaurante
{
    2 referencias
    public override Plato_Fuerte C_Plato_Fuerte() => new Bandeja_Paisa();
    2 referencias
    public override Bebida C_Bebida() => new Limonada_De_Coco();
    2 referencias
    public override Postre C_Postre() => new Arequipe();
}

1 referencia
class Restaurante_Tai : Restaurante
{
    2 referencias
    public override Plato_Fuerte C_Plato_Fuerte() => new PadThai();
    2 referencias
    public override Bebida C_Bebida() => new Te_Tailandes();
    2 referencias
    public override Postre C_Postre() => new Mango_Sticky_Rice();
}
```

```
1 referencia
class Restaurante_Med : Restaurante
{
    2 referencias
    public override Plato_Fuerte C_Plato_Fuerte() => new Gyro();
    2 referencias
    public override Bebida C_Bebida() => new Vino();
    2 referencias
    public override Postre C_Postre() => new Baklava();
}
2 referencias
public Menu()
{
    InitializeComponent();
}

1 referencia
private void Menu_Load(object sender, EventArgs e)
{

}

1 referencia
private void Mexicana_Click(object sender, EventArgs e)
{
    MostrarMenu(new Restaurante_Mex());

}


1 referencia
private void Japonesa_Click(object sender, EventArgs e)
{
    MostrarMenu(new Restaurante_Jap());

}

 5 referencias
 void MostrarMenu(Restaurante r)
{
    Servir s = new Servir(r);
    s.Show();
    this.Hide();
}

1 referencia
private void Cerrar_Click(object sender, EventArgs e)
{
    Entrada f1 = new Entrada();
    f1.Show();
    this.Close();
}
```

```
        1 referencia
        private void Colombiana_Click(object sender, EventArgs e)
        {
            MostrarMenu(new Restaurante_Col());
        }

        1 referencia
        private void Tailandesa_Click(object sender, EventArgs e)
        {
            MostrarMenu(new Restaurante_Tai());

        }

        1 referencia
        private void Mediterranea_Click(object sender, EventArgs e)
        {
            MostrarMenu(new Restaurante_Med());

        }

        1 referencia
        private void Mediterranea_Paint(object sender, PaintEventArgs e)
        {

        }

        1 referencia
        private void Mexicana_Paint(object sender, PaintEventArgs e)
        {

        }

        1 referencia
        private void Japonesa_Paint(object sender, PaintEventArgs e)
        {

        }

        0 referencias
        private void button1_Click(object sender, EventArgs e)
        {
        }

        1 referencia
        private void SALIR_Click(object sender, EventArgs e)
        {
            Application.Exit();
        }

        1 referencia
        private void panel3_Paint(object sender, PaintEventArgs e)
        {

        }
    }
}
```

Servicio

```
namespace Restaurante_Mix
{
    5 referencias
    public partial class Servir : Form
    {


        1 referencia
        public Servir(Menu.Restaurante r)
        {
            InitializeComponent();
            MostrarMenu(r);

        }
        1 referencia
        void MostrarMenu(Menu.Restaurante r)
        {
            Menu.Plato_Fuerte plato = r.C_Plato_Fuerte();
            Menu.Bebida bebida = r.C_Bebida();
            Menu.Postre postre = r.C_Postre();

            lblPlato.Text =  plato.Nombre();
            lblBebida.Text = bebida.Nombre();
            lblPostre.Text = postre.Nombre();
        }


        1 referencia
        private void Servir_Load(object sender, EventArgs e)
        {

        }

        1 referencia
        private void lblPostre_Click(object sender, EventArgs e)
        {

        }

        0 referencias
        private void Regresar_Paint(object sender, PaintEventArgs e)
        {

        }

        1 referencia
        private void panel2_Click(object sender, EventArgs e)
        {
            Despedida I = new Despedida();
            I.Show();
            this.Hide();
        }
```

```
        1 referencia
        private void panel3_Click(object sender, EventArgs e)
        {
            Menu Menu1 = new Menu();
            Menu1.Show();
            this.Hide();
        }

        1 referencia
        private void SALIR_Click(object sender, EventArgs e)
        {
            Application.Exit();
        }

        1 referencia
        private void panel2_Paint(object sender, PaintEventArgs e)
        {

        }
    }
}
```

Despedida

```
namespace Restaurante_Mix
{
    5 referencias
    public partial class Despedida : Form
    {
        1 referencia
        public Despedida()
        {
            InitializeComponent();
        }

        1 referencia
        private void SALIR_Click(object sender, EventArgs e)
        {
            Application.Exit();
        }

        1 referencia
        private void Despedida_Load(object sender, EventArgs e)
        {

        }
    }
}
```

Pruebas

ID	Descripción	Prioridad	Pasos	Resultado esperado	Resultado obtenido	Estado
1	Verificar el botón de cerrar el programa.	4	1. Abrir la aplicación. (Fig.1) 2. Cerrar la aplicación.	Que la aplicación y el programa se detengan	La app se cierra correctamente y deja de ejecutarse el programa	Correcto
2	Verificar el botón del menú.	5	1. Abrir la aplicación. (Fig.1) 2. Presionar el botón del menú en cualquier área. 3.Ver el menú disponible. (Fig.2)	Que se muestre la forma que contiene el menú sin que sea muy obvia la otra forma y que estén en la misma ubicación de la pantalla.	Se muestra bien, solo un pequeño cambio en el tamaño a diferencia de la ventana anterior, se muestra un desfase por el aumento del tamaño.	Correcto
3	Verificar el botón para ir atrás.	4	1. Abrir la aplicación. (Fig.1) 2. Ir al menú. 3. Regresar a la pantalla principal.	Que regrese a la pantalla anterior sin congelamientos o lag.	Se muestra el cambio del tamaño y la carga del icono.	Correcto
4	Verificar la elección del platillo en el menú.	5	1. Abrir la aplicación. 2. Ir al menú. 3. Seleccionar un platillo. 4. Ver los productos que pertenecen al estilo de comida. (Fig.3)	Que se muestre el nuevo platillo, bebida y postre correspondiente.	Se muestra bien el platillo, bebida y postre que pertenece a la familia.	Correcto
5	Verificar el cambio de platillo.	5	1. Abrir la aplicación. (Fig.1) 2. Ir al menú. 3. Seleccionar un platillo. 4. Ver los productos que pertenecen al estilo de comida. 5. Presionar “Ordenar nuevamente” y elegir otro estilo de comida. (Fig.2) 6. Visualizar los nuevos platillos. (Fig.3)	Que los nombres de los nuevos platillos se muestran correctamente y que se muestre la familia completa.	Al elegir otro platillo se vuelve a desplegar el menú y se puede elegir otro estilo de comida.	Correcto

Conclusión

Con el patrón fábrica abstracta podemos agregar más productos y familias de productos gracias al principio de abierto/cerrado sin que el cliente se vea afectado. Ya que la clase abstracta contiene los tipos de productos y cada clase concreta implementa su propia familia permite tener un bajo acoplamiento y facilita el poder agregar nuevos estilos de comida sin afectar el programa.